Spline-Based Trajectory Generation and Stanley Tracking for Differential-Drive Navigation in ROS 2

Email: raybimlendra@gmail.com

Keywords:

1 Introduction

Reliable path tracking is a central requirement for mobile robots operating in structured indoor and laboratory environments. In this work we present a compact ROS 2 navigation stack for a differential-drive robot simulated in TurtleBot3 Gazebo. The package focuses on the full path-following pipeline: waypoint generation, geometric path smoothing, time-parameterized trajectory generation, closed-loop tracking, and quantitative performance logging.

The implementation is intentionally lightweight and modular. Waypoints are produced from predefined path presets, test-track layouts. A spline-based smoother converts sparse waypoints into a dense reference path. A trajectory generator assigns headings and timestamps to each sampled point, and a controller combines heading regulation with a Stanley-style cross-track correction term [1]. Finally, a metrics logger records tracking error and velocity signals for offline evaluation.

The goal of this report is to document the design of the pipeline and summarize one representative simulation run using the default `test_track` preset in closed-loop mode.

2 Data and Methods

2.1 System architecture

The navigation package is implemented as an `ament_python` ROS 2 package and uses `rclpy`, `geometry_msgs`, and `nav_msgs`. The launch file starts the following nodes:

1. `waypoint_publisher`, which publishes the selected waypoint preset as a `PoseArray`;
2. `path_smoother`, which generates a dense `nav_msgs/Path`;
3. `trajectory_generator`, which time-parameterizes the path;
4. `trajectory_controller`, which computes velocity commands from odometry feedback;
5. `navigation_metrics_logger`, which stores performance measurements to CSV.

The experiments reported here were produced with the default launch configuration:
`path_preset=test_track`,
`closed_path=true`,
`sample_spacing=0.08 m`,
`spline_smoothing=0.02`,
`cruise_speed=0.3 m/s`,
`acceleration=0.5 m/s`^{2},
`max_linear_velocity=0.16 m/s`, and `max_angular_velocity=2.2 rad/s`.

2.2 Waypoint generation and smoothing

Each path preset returns a set of planar waypoints (x_i, y_i), shifted so that the first waypoint becomes the origin. This simplifies initialization in simulation. To obtain a continuous path, the waypoints are smoothed with SciPy's parametric spline tools `splprep` and `splev` [2]. The resulting dense curve is then resampled at approximately uniform arc-length spacing:

$$s_k = k\Delta s, \qquad k = 0, \ldots, N-1, \tag{1}$$

where Δs is the user-defined sample spacing. Path headings are estimated from neighboring points using finite differences. For closed paths, the heading computation wraps around the first and last samples.

2.3 Timed trajectory generation

The smoothed path is converted into a timed trajectory by attaching a heading and a timestamp to each sampled point. For open paths, the software supports constant-speed and trapezoidal-speed timing. For the closed-loop experiment reported here, the implementation uses constant nominal timing around the loop, so the timestamp of point i is computed from the cumulative arc length d_i as

$$t_i = \frac{d_i}{v_c}, \tag{2}$$

where v_c is the cruise speed. This produces a reference speed profile that is approximately uniform along the track.

2.4 Trajectory tracking

The controller uses the robot odometry and projects the control point to the front axle position:

$$x_f = x + L\cos\psi, \qquad y_f = y + L\sin\psi, \tag{3}$$

where (x, y, ψ) is the robot pose and L is the front-axle offset. The nearest path segment is selected, and the controller computes the heading error e_ψ and cross-track error e_c. The commanded angular velocity follows a Stanley-style law:

$$\omega = k_h e_\psi + \arctan\left(\frac{k_s e_c}{v + \epsilon}\right), \tag{4}$$

where k_h is the heading gain, k_s is the Stanley gain, v is the commanded linear speed, and ϵ is a softening term used to avoid division by zero at low speeds. The linear speed command is regulated toward the reference speed while reducing speed under large heading errors:

$$v = \mathrm{sat}\left(v_{\text{cur}} + k_v\left(\alpha v_{\text{ref}} - v_{\text{cur}}\right)\right), \tag{5}$$

where v_{cur} is the measured linear speed, k_v is a speed gain, $\mathrm{sat}(\cdot)$ enforces velocity limits, and $\alpha \in [0.15, 1]$ is a heading-dependent scaling factor.

2.4.1 Why we used Stanley Controller ?

Stanley controller is good kinematic model used for Car path following at low speed so I thought it can be used over here to. With some help of ChatGPT, I tweaked the model and from there, ω, is computed using Stanley and also created a relation of the heading error with the velocity so that velocity will be dropped when heading error is high and will have enough time to correct the yaw rate.

2.5 *Evaluation protocol*

The metrics logger stores, at each odometry update, the robot position, nearest reference point, path error, heading error, linear velocity, and angular velocity. The CSV file generated in the current workspace contains 5265 samples over a run lasting 105.28 s. Summary statistics reported below were computed from `results/navigation_metrics.csv`.

3 Results

The default configuration produced stable path following on the `test_track` loop. The mean path-tracking error was 0.0226 m and the root-mean-square error was 0.0297 m, indicating that the robot remained close to the spline reference for most of the experiment. The maximum path error was 0.0846 m, remaining below 10 cm throughout the recorded run.

Heading regulation is good in the metrics log. The mean absolute heading error was $4.08°$, while the maximum absolute heading error reached $48.44°$ when starting from the initial position. This indicates that most of the run stayed well aligned with the local path tangent, with larger deviations concentrated in transient recovery periods and sharper curvature transitions.

The velocity signals also reflect the controller constraints. The mean linear velocity was 0.1138 m/s with a peak of 0.1599 m/s, staying close to the configured linear limit. The mean absolute angular velocity was 0.1025 rad/s, and the peak angular velocity reached the configured saturation of 2.2 rad/s.

metric	value	unit
Duration	123.78	s
Number of samples	6190	–
Mean path error	0.0226	m
Path error RMSE	0.0297	m
Max path error	0.0846	m
Mean abs. heading error	4.08	deg
Max abs. heading error	48.44	deg
Mean linear velocity	0.1138	m/s
Max linear velocity	0.1599	m/s
Mean abs. angular velocity	0.1025	rad/s
Max abs. angular velocity	2.2000	rad/s

Table 1: **Navigation performance on the default closed-loop test track**. Statistics computed from the recorded ROS 2 navigation metrics log.

4 Conclusion

This report presented a ROS 2 navigation pipeline for differential-drive trajectory tracking in simulation. The system combines predefined path generation, spline-based smoothing, time-parameterized trajectory generation, Stanley-inspired steering correction, and automatic metric logging. In the default `test_track` experiment, the method achieved very small maximum path error and approximately 0.0297 cm RMS tracking error over more than 100 s of closed-loop operation.

The current implementation is modular enough to support further extensions. Immediate next steps include curvature-aware speed planning for closed paths, comparison against **MPC controllers**, and evaluation across multiple waypoint presets and noise conditions along with **Obstacle avoidance**. The package therefore provides a practical baseline for reproducible robot-navigation experiments in ROS 2.

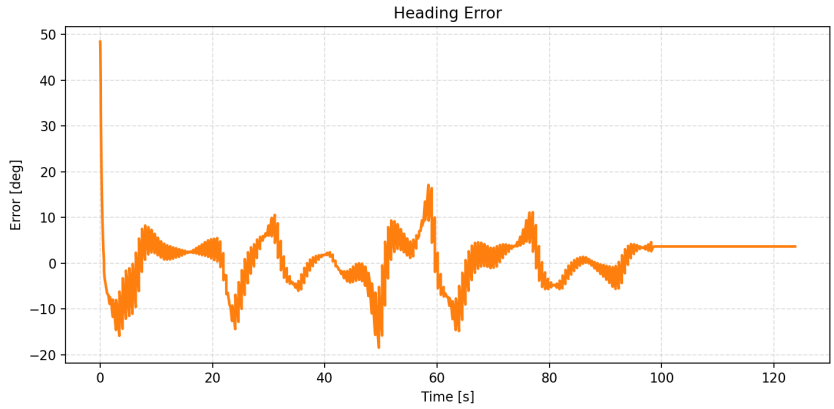

Figure 1: Heading error

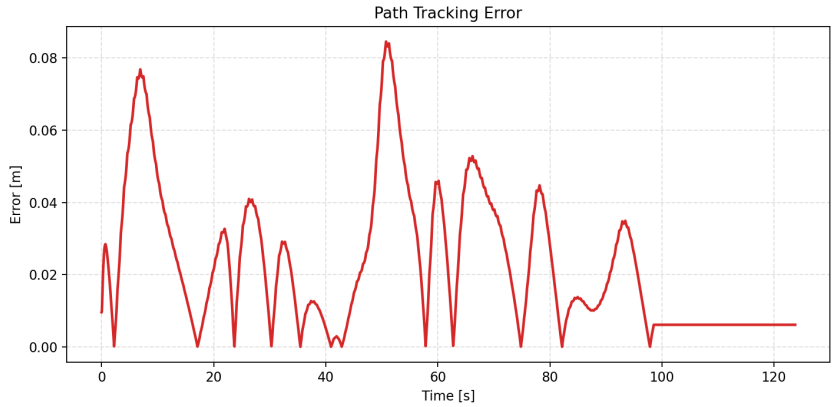

Figure 2: Path Tracking Lateral error

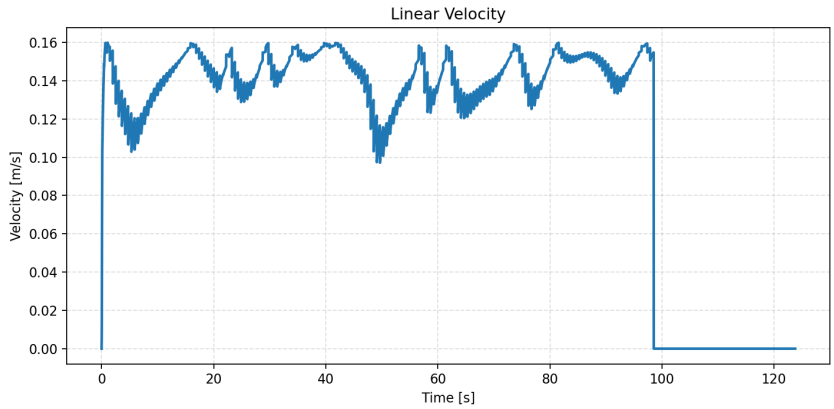

Figure 3: Linear Velocity (Longitudinal)

Future Work And Extension

The future work will be to implement the MPC controller along with Stanley controller (as low level controller) for perfect trajectory tracking. In this case, we have generated the velocity profile but that is completely based on maximum acceleration and deceleration and time based. However, it can be optimized based on the curvature of the path, which will give us the best possible speed profile while taking model dynamics into consideration. To implement in real robots, it requires constraint and need to obey the safety rules and regulation, MPC or other

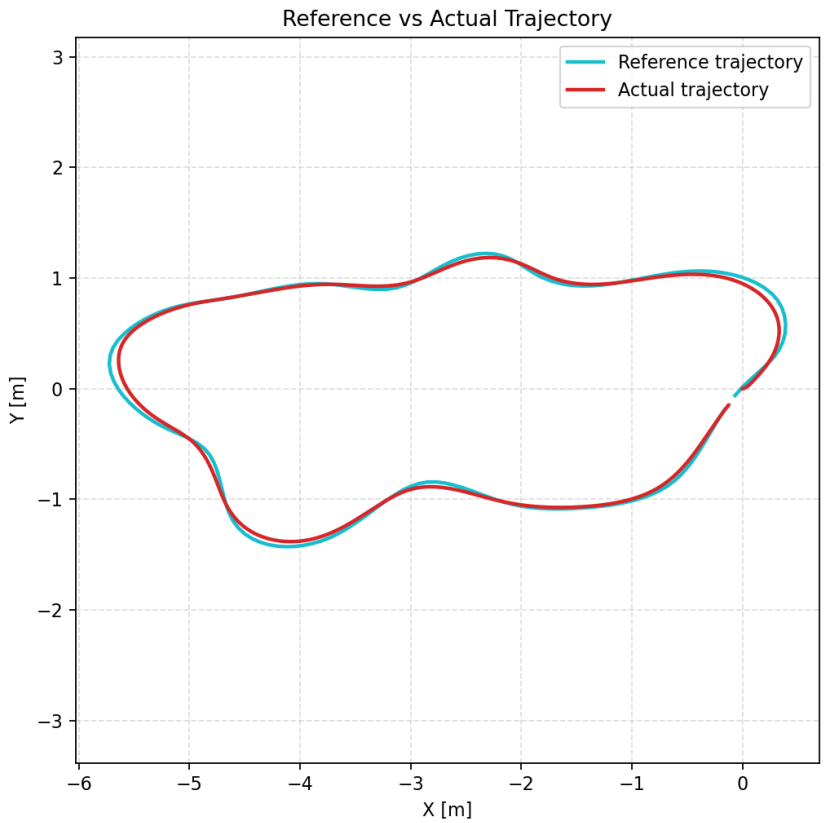

Figure 4: Trajectory of the Turtlebot3

type of advance controller like MPC is required which is best fit with the real world constraints. Moreover, a real robot can have time-delay so one have to take that thing into considerations in to the model and controller and have to choose control action wisely. Disturbance can be other variable for real world for which robustness should be analyzed for disturbance distribution and find the optimal control action which will ensure safety and constraint satisfaction.

Acknowledgments

The authors acknowledge the open-source ROS 2, TurtleBot3, NumPy, SciPy, and Matplotlib communities for the software infrastructure used in this work.AI tools (ChatGPT is used for the helper function generation like waypoint and path generation) and test code.

Availability of data and software code

The source code, launch files, unit tests, metrics CSV, and generated plots are available in the github at `https://github.com/bhimray/origin_ws.git`. The main package is located in `src/origin_navigation`, while the evaluation output used in this report is stored in `results/navigation_metrics.csv` and `results/navigation_plots/`.

References

[1] Sebastian Thrun, Michael Montemerlo, Hendrik Dahlkamp, David Stavens, Andrei Aron, James Diebel, Philip Fong, John Gale, Morgan Halpenny, Gabriel Hoffmann, Kenny Lau, Celia Oakley, Mark Palatucci, Vaughan Pratt, Pascal Stang, Sven Strohband, Cedric Dupont, Lars-Erik Jendrossek, Christian Koelen, Charles Markey, Carlo Rummel, Joe van Niekerk, Eric Jensen, Philippe Alessandrini, Gary Bradski, Bob Davies, Scott Ettinger, Adrian Kaehler, Ara Nefian, and Pamela Mahoney. Stanley: The robot that won the DARPA grand challenge. *Journal of Field Robotics*, 23(9):661–692, 2006.

[2] SciPy Community. SciPy `splprep` documentation. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.splprep.html`, 2026. Accessed March 13, 2026.